



Attention Is All You Need

BOAZ

24기 분석 김가영 배현진

CONTENTS

1

Before
Transformer



2

Model
Architecture



3

Attention



4

Positional
Encoding



5

Residual
Connection &
Feedforward
Layer



“ BERT는 Transformer의
Encoder 부분을,
GPT는 Transformer의
Decoder 부분을 활용한 모델 ”

Chapter 0

Attention Is All You Need

Attention

A

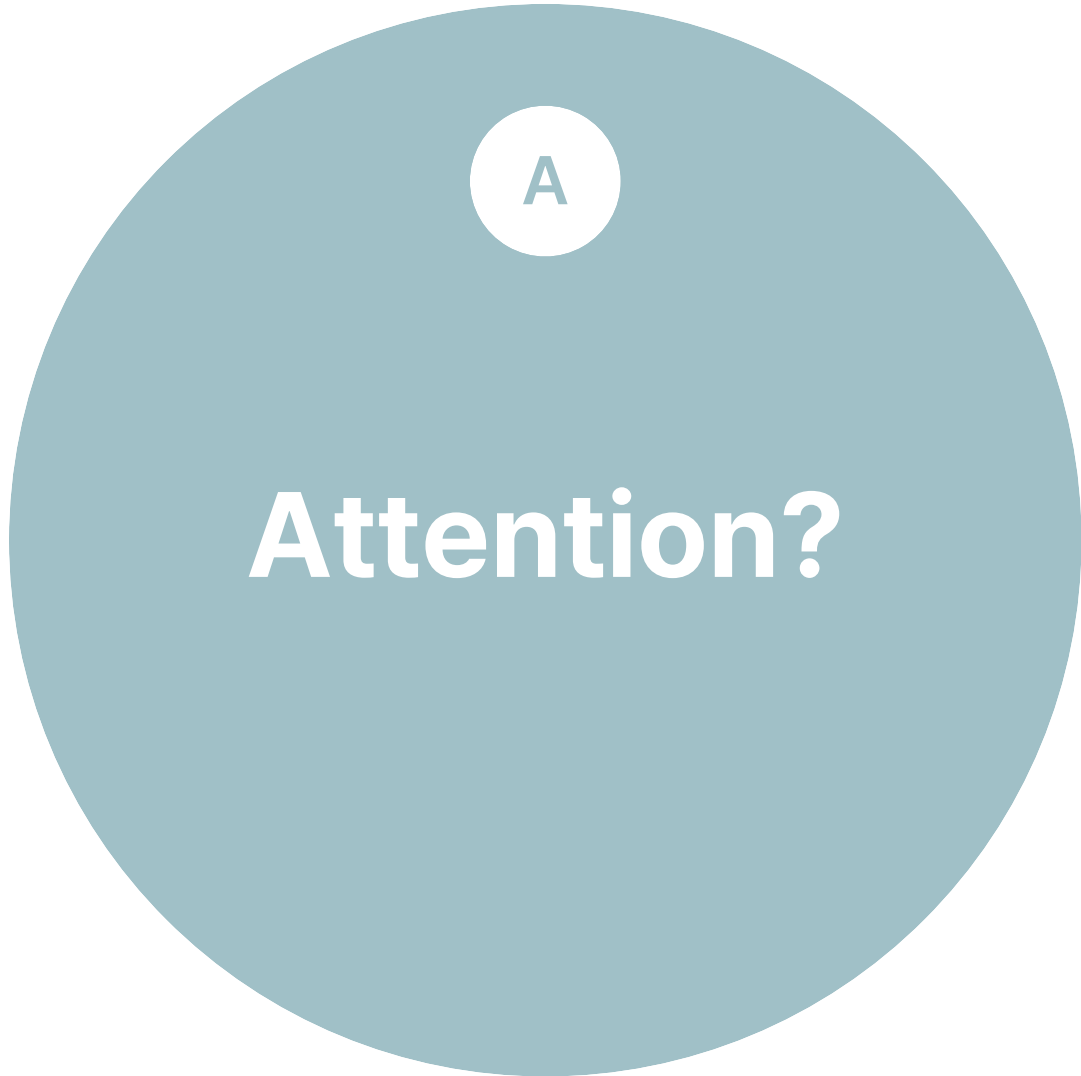
기존 인코더 디코더의 성능을 강화시키며
이미 주목받고 있던 메커니즘

B

Attention을 최초로 발표한 것 X

C

Attention만으로도 입력 데이터에서
중요한 정보들을 찾아내 단어를 인코딩 할 수 있다!



Chapter 1

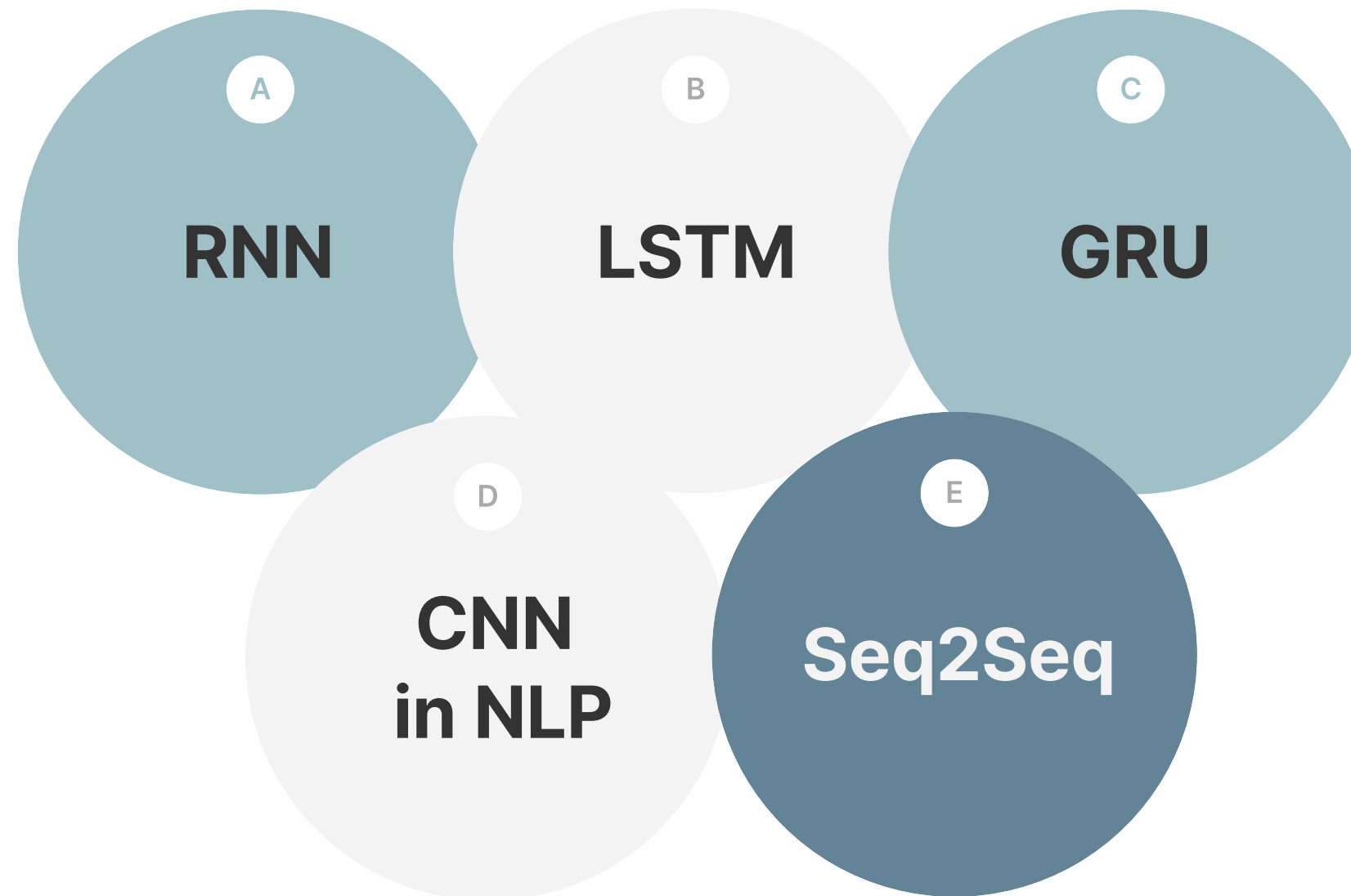
Before the Transformer..



A	RNN	B	LSTM
C	GRU	D	CNN in NLP
E	Seq2Seq	F	Why transformer

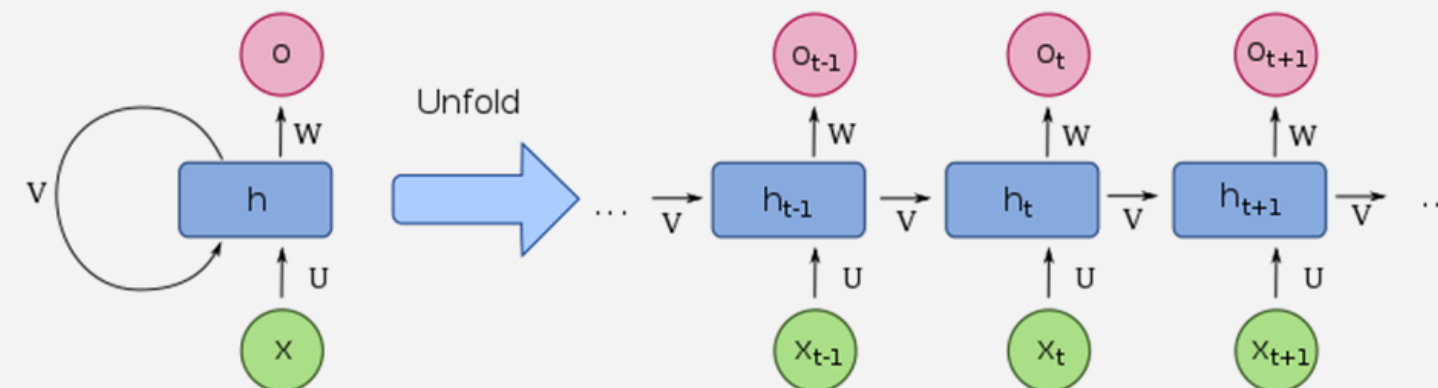
Sequence Modeling

어떠한 Sequence를 가지는 데이터로부터
또 다른 Sequence를 가지는 데이터를 생성하는 task



Recurrent Neural Networks (RNN)

- (이전 모델) FNN: 입력을 한 번에 받아서 바로 출력 $\rightarrow$ 독립적인 데이터만 학습 가능
- RNN: 시퀀스 데이터(문장, 음성 등)를 처리하기 위해 순차적 연산을 수행하는 모델
 - 이전 타임스텝의 정보를 현재 타임스텝으로 전달하는 **hidden state** 구조를 가짐



Recurrent Neural Networks (RNN)

- 문맥을 고려한 정보 처리 가능 → 시퀀스 데이터에서 유용
- But 긴 문장에서 기울기 소실 및 폭발 문제로 인한 **장기 의존성** 문제 발생
 - 장기 의존성(Long-Term Dependency): 모델이 멀리 있는 단어 간의 관계를 학습하지 못하는 상황

기울기 소실(Gradient Vanishing) 문제란?

- 역전파 과정에서, RNN의 가중치 업데이트를 위한 그래디언트가 계속 곱해질 때 아주 작은 값이 되어 사라지는 현상
- 특정 단어가 너무 먼 위치에 있을 경우, 초기 단어의 정보가 소실되어 모델이 제대로 학습되지 않음
- $\longleftrightarrow$ 기울기 폭발(Gradient Explosion)
 - 너무 큰 기울기가 곱해지면서 손실 값이 비정상적으로 커지는 현상

Why?

A

RNN의 순환 구조에서는

$$\frac{\partial L}{\partial W} = \prod_{t=1}^T \frac{\partial h_t}{\partial h_{t-1}} \text{ 형태로 미분이 누적됨}$$

B

만약 $\frac{\partial h_t}{\partial h_{t-1}}$ 이 1보다 작은 값이라면 계속 곱해질수록 그래디언트 값이 0에 가까워짐
→ Gradient Vanishing

C

반대로 1보다 큰 값이면 폭발적으로 증가함
→ Gradient Explosion

D



장기 의존성 문제

기울기 소실이 발생하면, 과거 정보가 학습되지 않음
→ 모델이 멀리 있는 단어 간의 관계를 학습하지 못함

Long Short-Term Memory (LSTM)

- 중요한 정보는 직접 기억하고, 불필요한 정보는 잊도록 설계
- Cell State: 중요한 정보는 "직접" 경로를 통해 손실 없이 유지됨
- Forget, Input, Output 게이트를 활용
 - Forget Gate: 과거의 정보를 버릴지 유지할지 결정하는 게이트
 - Input Gate: 새로운 정보 추가 여부를 결정하는 게이트
 - Output Gate: 다음 Hidden State를 생성하는 게이트 → 최종 출력 값은 활성화 함수를 통과한 후 결정

Long Short-Term Memory (LSTM)

- Forget Gate → Input Gate → Cell State 업데이트 → Output Gate
- 기울기 소실 문제 완화, 장기 의존성 문제 개선
- But 구조가 복잡하고 연산량이 많아 학습 속도가 느림

Gated Recurrent Unit (GRU)

- Reset Gate, Update Gate 두 개의 게이트만 사용
 - Reset Gate: 과거 정보를 삭제할지 결정
 - Update Gate: LSTM의 Forget Gate + Input Gate 역할, 과거 정보를 얼마나 유지할지 결정
- Cell State 대신 Hidden State 하나만 사용
 - Candidate Hidden State: 현재 입력과 조합된 새로운 잠재 상태 생성
 - 최종 Hidden State: Update Gate의 결정에 따라 최종 상태 계산

Gated Recurrent Unit (GRU)

- Reset Gate → Candidate Hidden State 계산 → Update Gate → Hidden State 업데이트
 - 별도의 Output Gate X
- LSTM보다 구조가 단순하여 연산량 감소

LSTM & GRU 한계

- RNN 기반이므로 병렬 연산 불가능 → 학습 속도 저하
- 긴 문장에서 여전히 장기 의존성 문제 발생 가능

Convolutional Neural Networks (CNN)

- 합성곱 필터를 사용해 국소적인 특징을 추출하는 모델
- 단어 단위로 합성곱 연산을 수행하여 특징을 학습
- 병렬 연산이 가능하므로 RNN보다 빠름
- But 커널 크기에 제한을 받음(n -gram) → 장기 의존성 학습이 어려움

Convolutional Neural Networks (CNN)

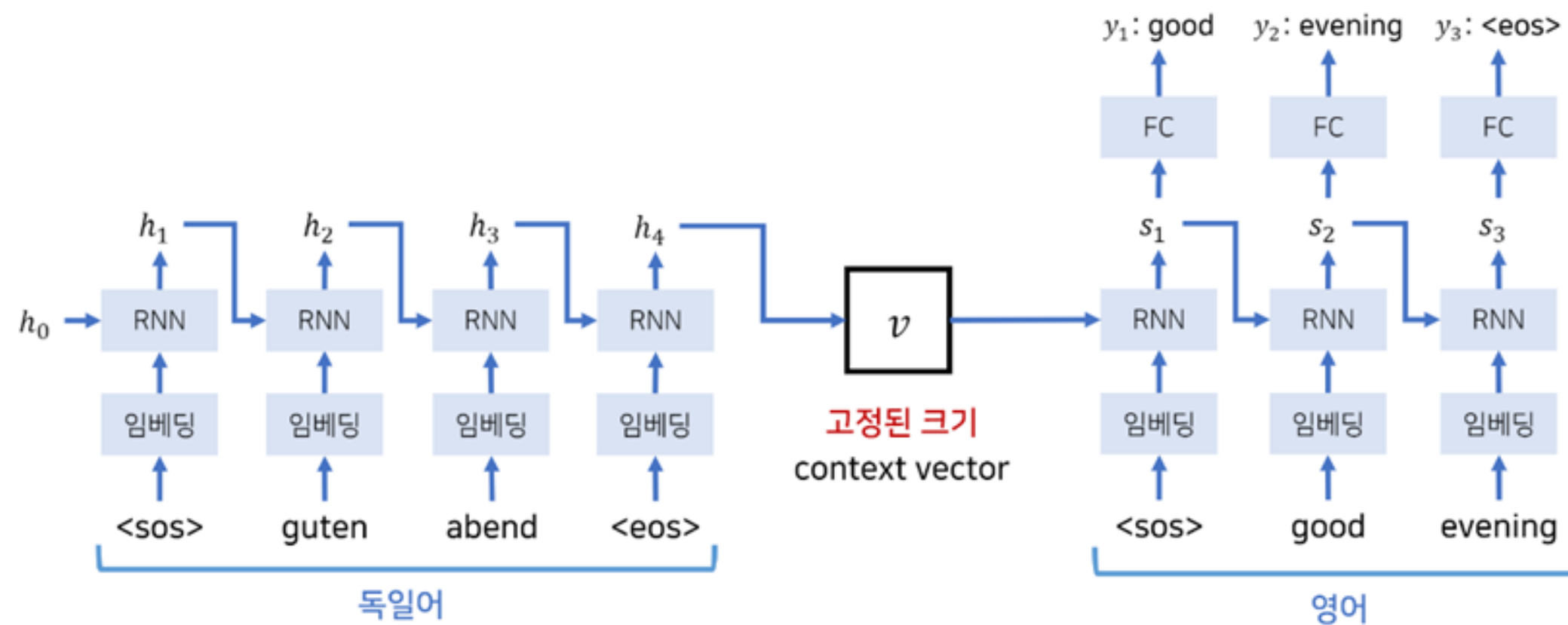
- 발전된 CNN 기반 모델
 - ByteNet
 - RNN 없이 CNN만으로 byte 단위 입력 시퀀스 처리
 - 1D dilated convolution + 각 층에 잔차 연결(Residual Connection) 적용
 - ConvS2S (Convolutional Sequence to Sequence)
 - CNN + Seq2Seq 구조를 사용하여 문장을 인코딩/디코딩
 - 여러 개의 CNN 레이어를 쌓아서 계층적으로 문맥 확장 (Hierarchical Feature Extraction)

Convolutional Neural Networks (CNN)

- But 문장의 길이가 길어질수록 커널 크기를 키우거나 계층을 깊게 쌓아야 함
 - ConvS2S에서는 선형적으로 증가
 - ByteNet에서는 로그적으로 증가 → ConvS2S보다 효율적이지만 여전히 거리 의 의존성이 존재
 - Transformer는 상수 수준의 연산량으로 구현 가능
- 장기 의존성 문제 해결 X

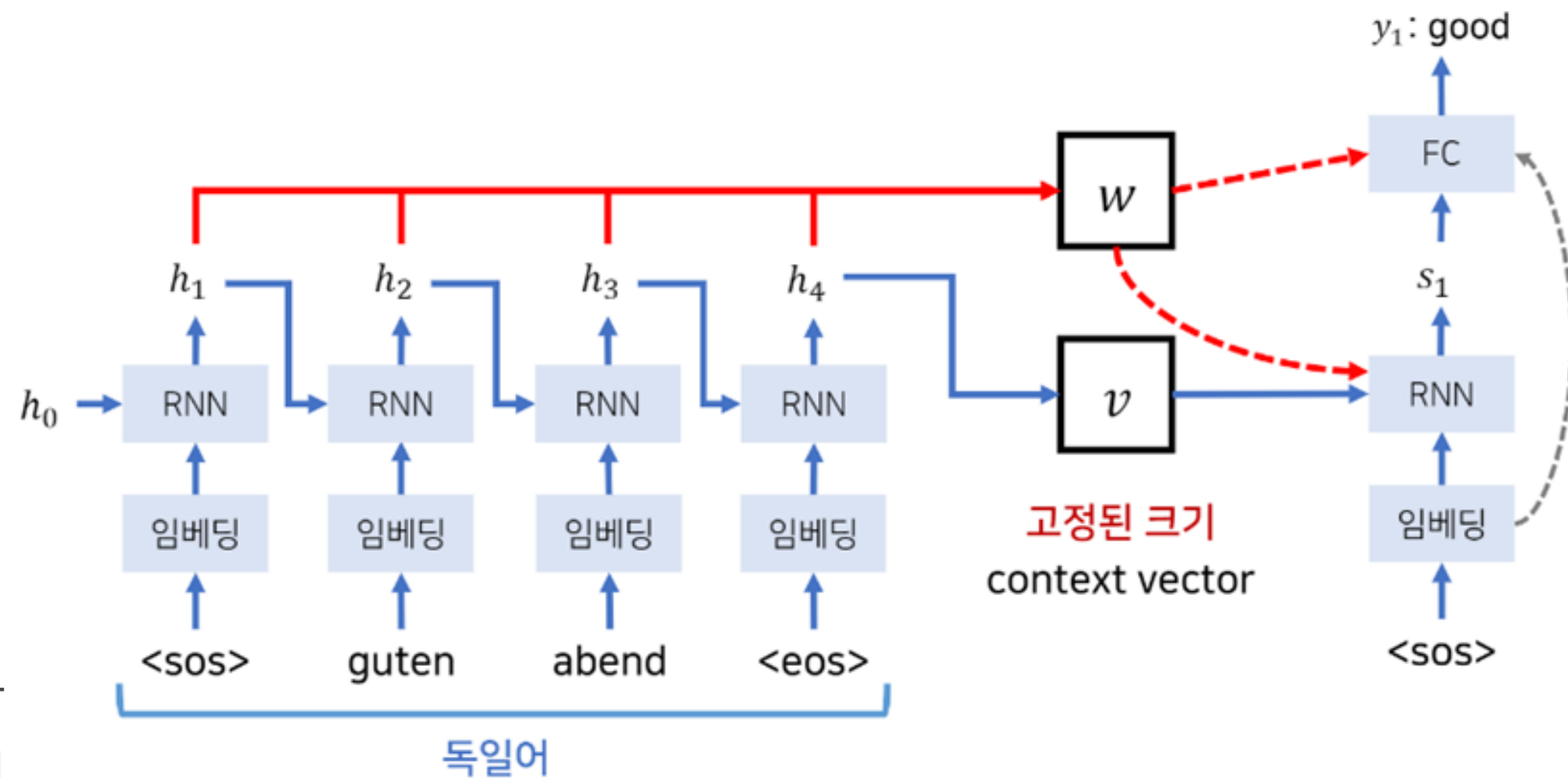
Seq2Seq

- Encoder-Decoder 구조로 구성되며, 일반적으로 RNN 기반(LSTM, GRU 등)으로 구현
 - Encoder: 입력 시퀀스를 압축된 벡터(= Context Vector)로 변환
 - Decoder: 이 벡터를 기반으로 출력 시퀀스를 생성
- 하나의 Context Vector에 소스 문장의 정보를 압축 → 병목 현상을 발생시켜 성능 하락의 원인

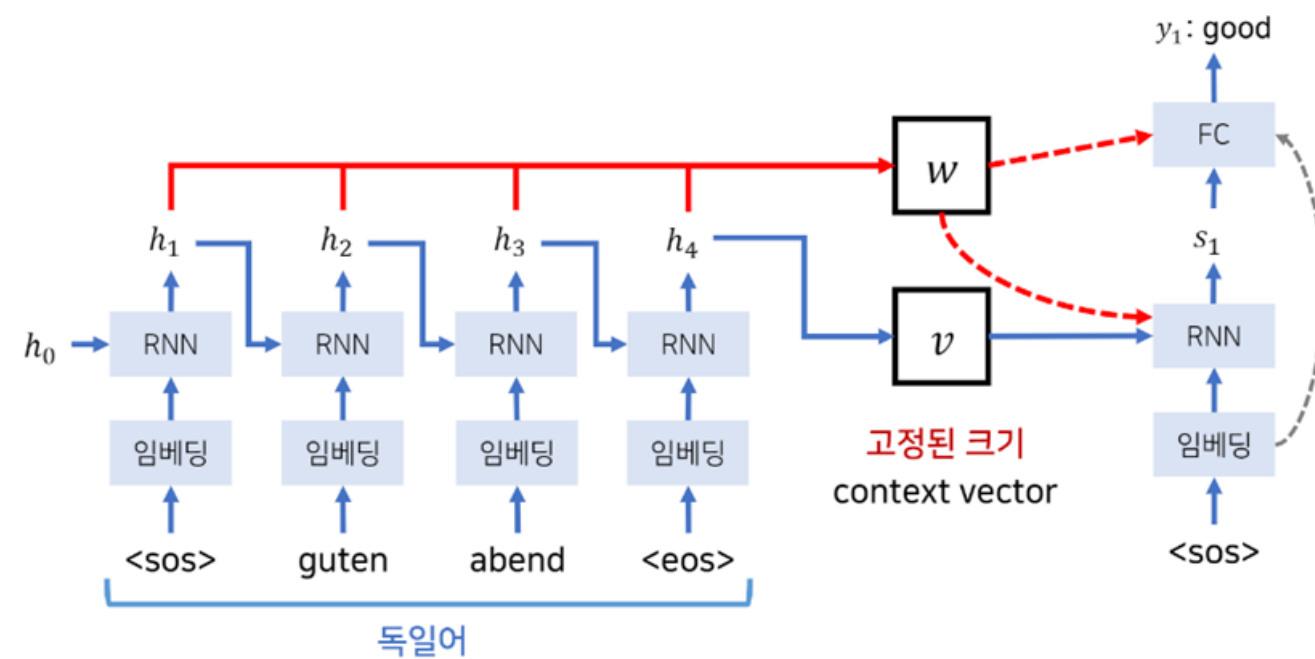


Seq2Seq with Attention

인코더의 모든 출력값들($h_1, h_2, h_3, \dots$)을 별도의 배열에 기록한 후, 디코더에서 출력 단어를 생성할 때 참고해서 입력 문장 전체를 반영할 수 있게 하자!



Seq2Seq with Attention



A

이전의 hidden state인 s_1 과 소스 문장에서의 hidden state 값들($h_1, h_2, \dots$)을 서로 묶어서 행렬곱 수행해 각각의 에너지 값(attention score)을 만든다.

B

에너지 값에 softmax함수를 취해서 확률값으로 바꿔 소스문장 각각의 hidden state 값 ($h_1, h_2, h_3, \dots$)에 대해서 어떤 벡터에 더 많은 가중치를 줄 지 참고한다.

C

가중치를 다 hidden state 값에 곱해주고 그 값을 모두 합해 $\text{weighted sum}(W)$ 을 구한다.

D

weighted sum 를 매번 출력 단어를 만들 때 반영한다.

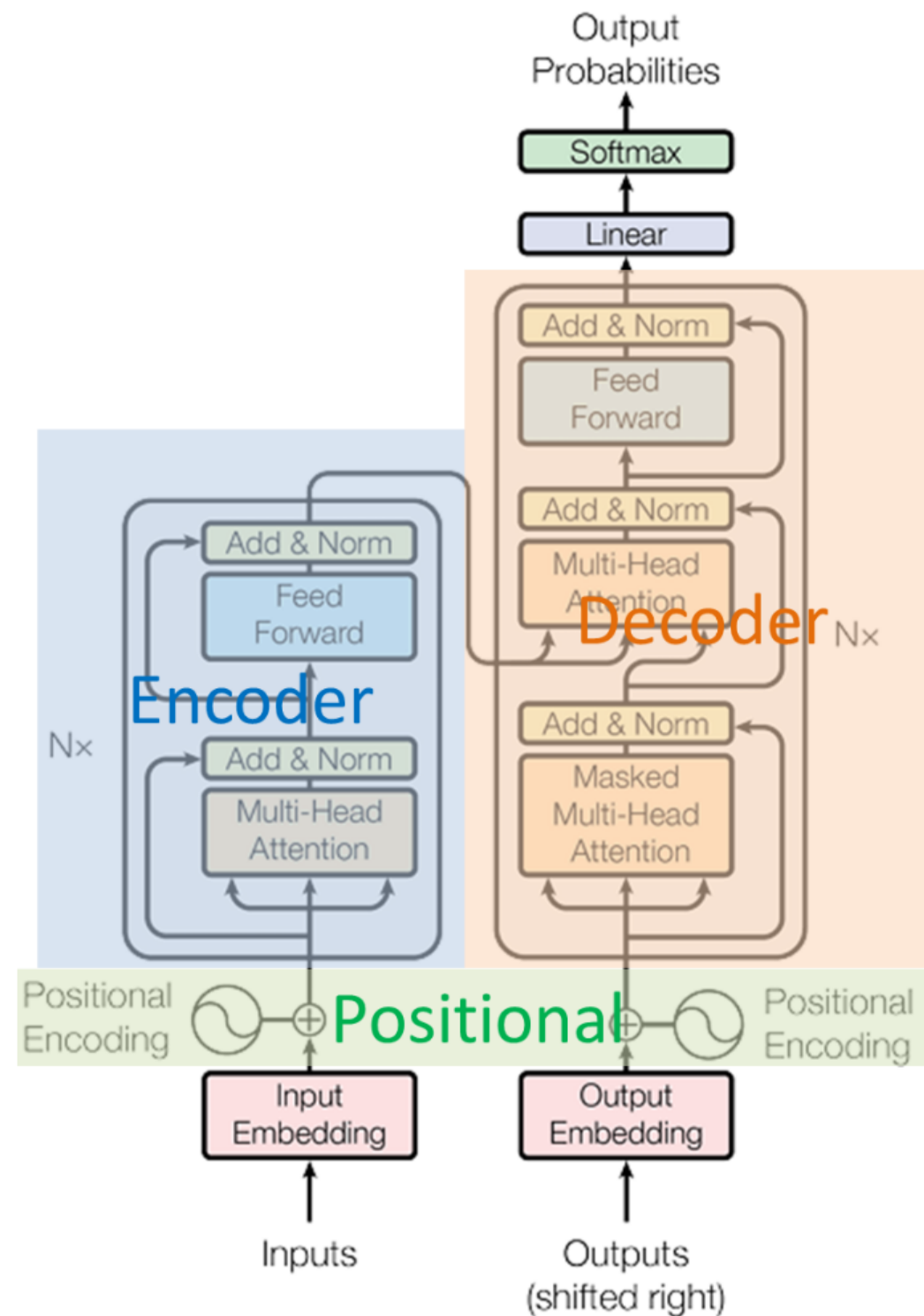
s_2 구하는 방법

Seq2Seq with Attention

- Attention을 Decoder에서만 한 번 사용
 - Decoder가 단어를 생성할 때마다, Encoder의 Hidden State와 비교하여 중요도를 결정하는 과정에서만 Attention이 동작
- Transformer는 Encoder와 Decoder에서 각각 여러 번 Attention을 수행
 - Self-Attention 매커니즘

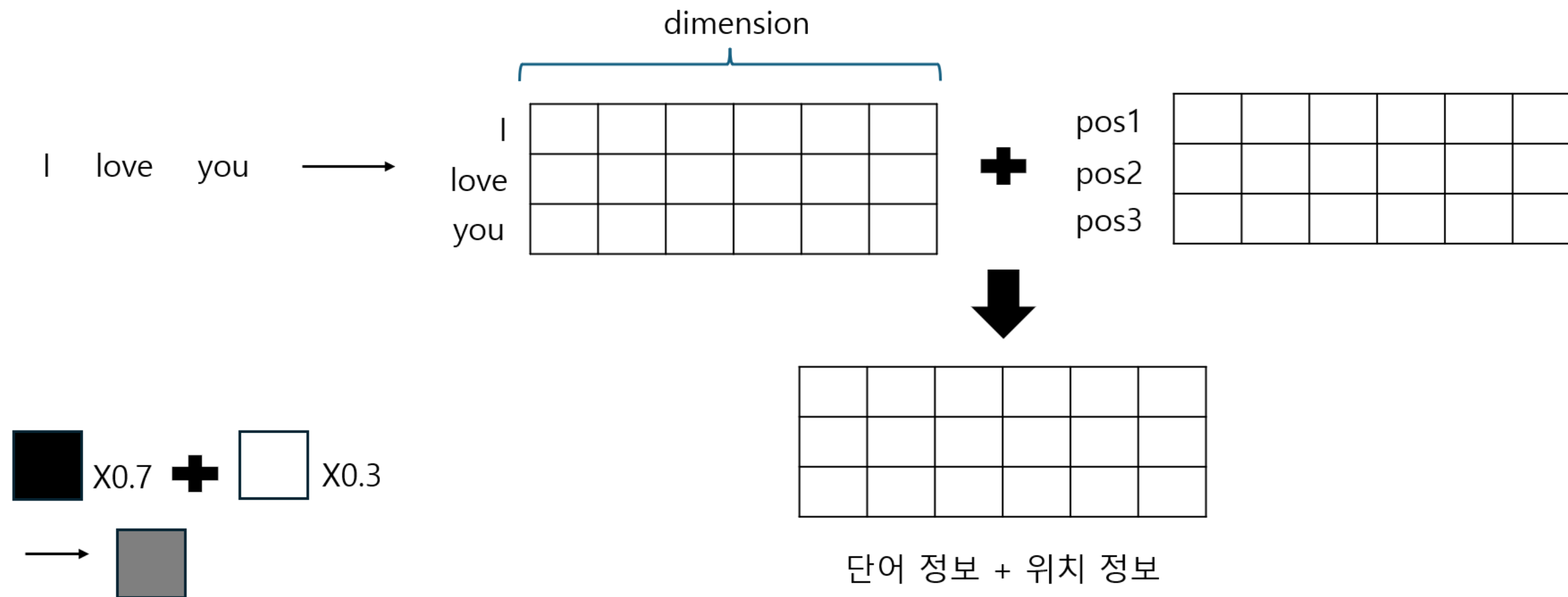
Why Transformer?

 연산 방식	Before 순차적	After 병렬적
 장기 의존성	Before 해결 X	After Self-Attention, Multi-Head Attention 으로 해결
 연산량	Before 선형적 ~ 로그적	After 상수 수준



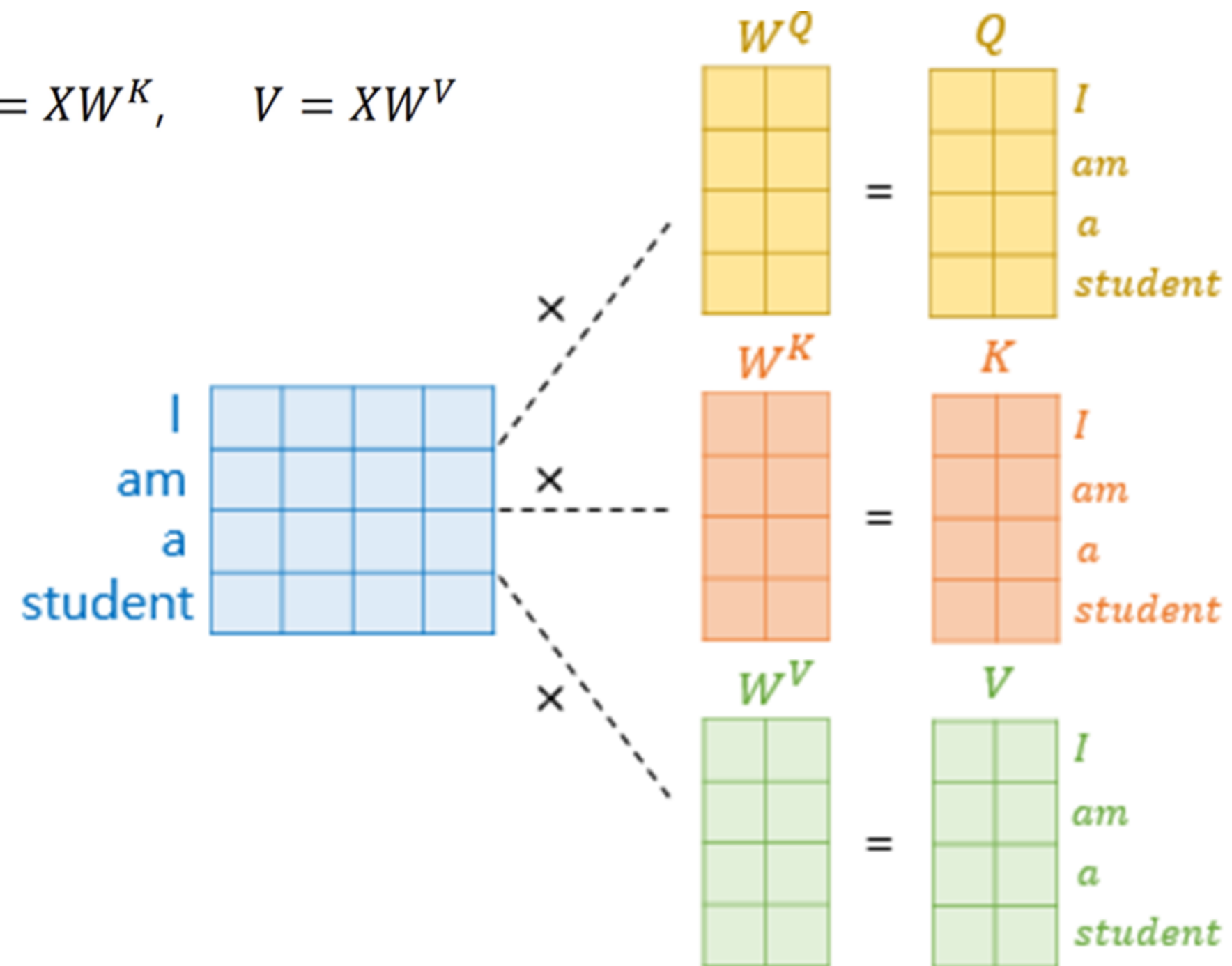
- auto-regressive
- Positional encoding
- Encoder
 - 2 sub-layers : multi-head self attention & feed-forward Net
 - residual connection
- Decoder
 - 3 sub-layers : multi-head self attention & encoder-decoder attention & feed-forward Net
 - residual connection

- 문장의 토큰(token)화
- 토큰의 벡터화 :
벡터화는 대규모 데이터셋에 의해서 사전 훈련된 임베딩 모델에 의해 이루어지기 때문에 이 임베딩 모델이 성능이 뛰어나야 Attention의 성능도 높아진다.
- Positional Encoding

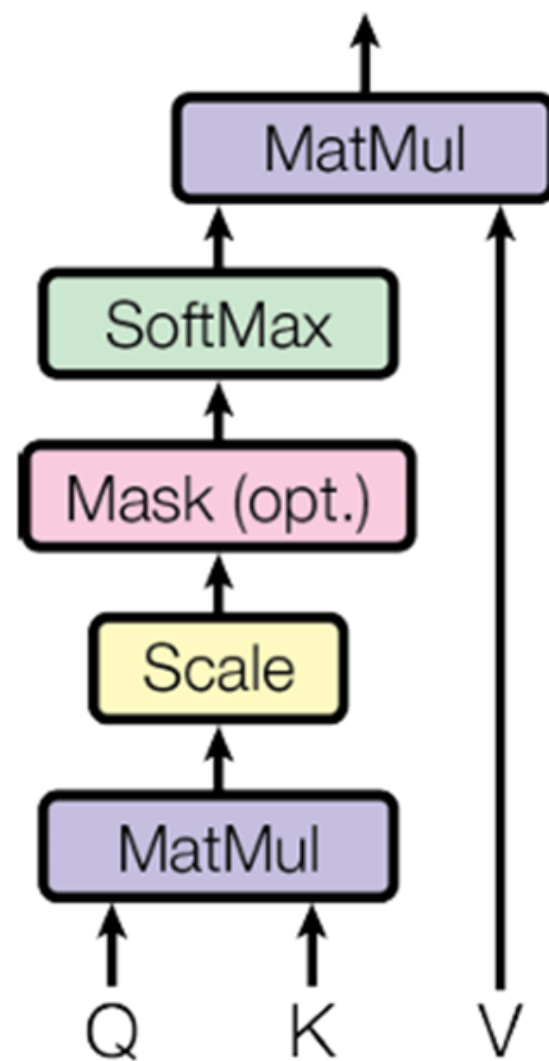


- Q, K, V의 시작 값이 동일하다

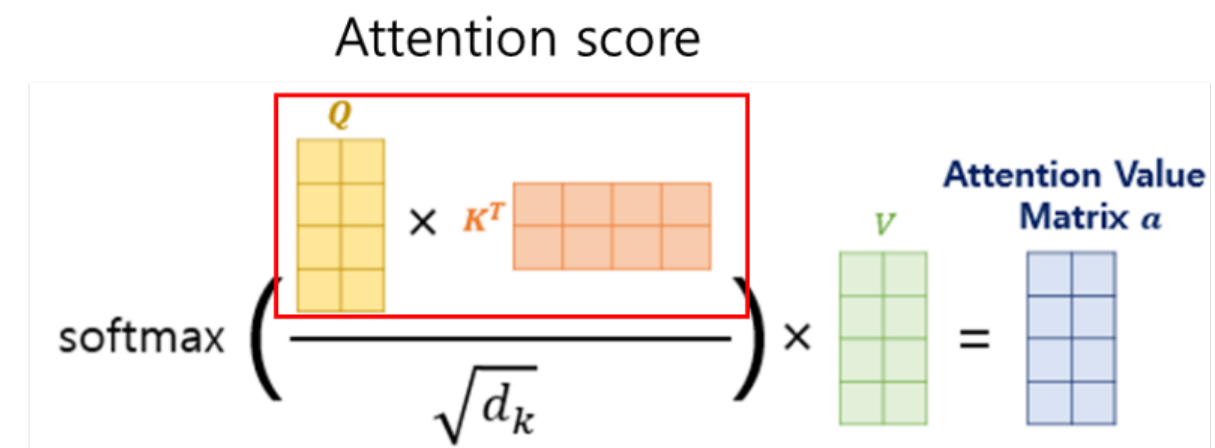
$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$



Scaled Dot-Product Attention



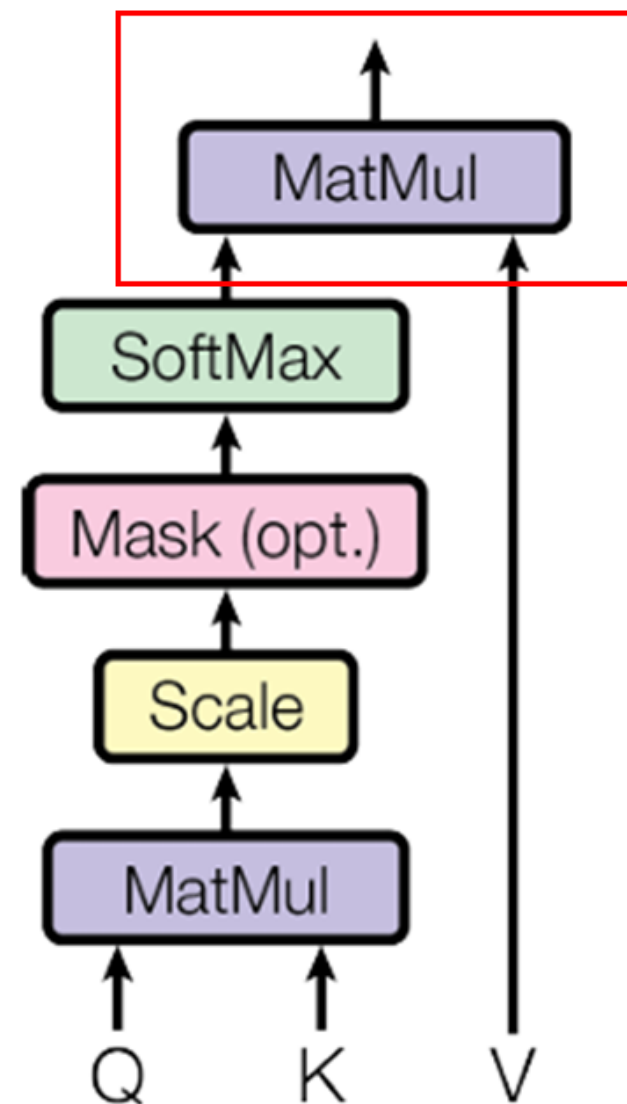
$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



	I	am	a	student
I	1.5	0.5	0.3	1.0
am	0.5	1.5	0.4	0.2
a	0.3	0.4	1.5	0.1
student	1.0	0.2	0.1	1.5

- A_{ij} 는 단어 단어 j와 얼마나 관련이 있는지를 나타내는 값
- 이를 softmax로 정규화 하면, 한 단어가 다른 단어로 부터 정보를 얼마나 가져갈지 결정하는 '가중치 행렬'이 됨
- $\sqrt{d_k}$ 는 softmax의 분포가 퍼지지 않게 하기 위한 값

Scaled Dot-Product Attention



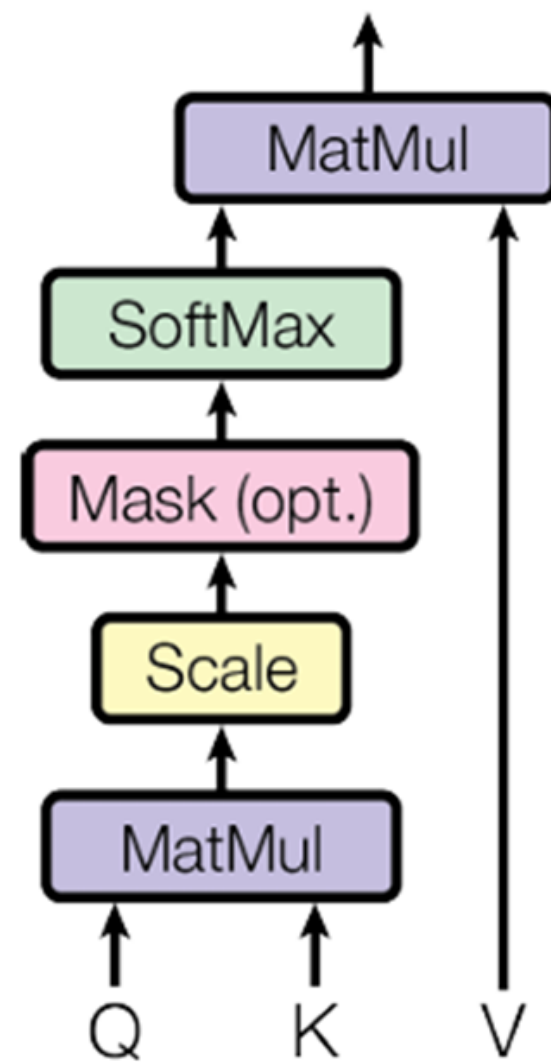
$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V = \text{Attention Value Matrix } a$$

The equation shows the calculation of the Attention Value Matrix a . It involves the dot product of Q and K^T , scaled by $\sqrt{d_k}$, passed through a softmax function, and then multiplied by V. The result is the Attention Value Matrix a .

- 각 단어가 다른 단어로부터 정보를 받아서 최종적인 새로운 표현을 만들게 됨.
- 어떤 단어는 자기 자신만 참고하고, 어떤 단어는 다른 단어에서 중요한 정보를 받아들일 수도 있음.
- 즉, 문맥(Context)이 반영된 새로운 단어 벡터를 생성하는 과정

- Value와의 행렬곱을 통해 "새로운 표현(representation)" 생성
- 어텐션 가중치 $\alpha(Q \cdot V^T)$ 를 V 에 곱하면, 각 단어가 다른 단어로부터 정보를 얼마나 반영할지를 결정함.
- 예를 들어, "I"의 최종 표현을 계산한다고 해보자.
 - $Z = 0.1 \cdot V_I + 0.8 \cdot V_{\text{love}} + 0.2 \cdot V_{\text{you}}$
 - 즉, "I"의 새로운 표현은 자기 자신(V_I)뿐만 아니라 "love"와 "you"의 정보를 비율에 맞게 반영한 값으로 업데이트됨.
 - "love"와의 유사성이 크기 때문에, "I"는 "love"의 정보를 더 많이 반영함.

Scaled Dot-Product Attention



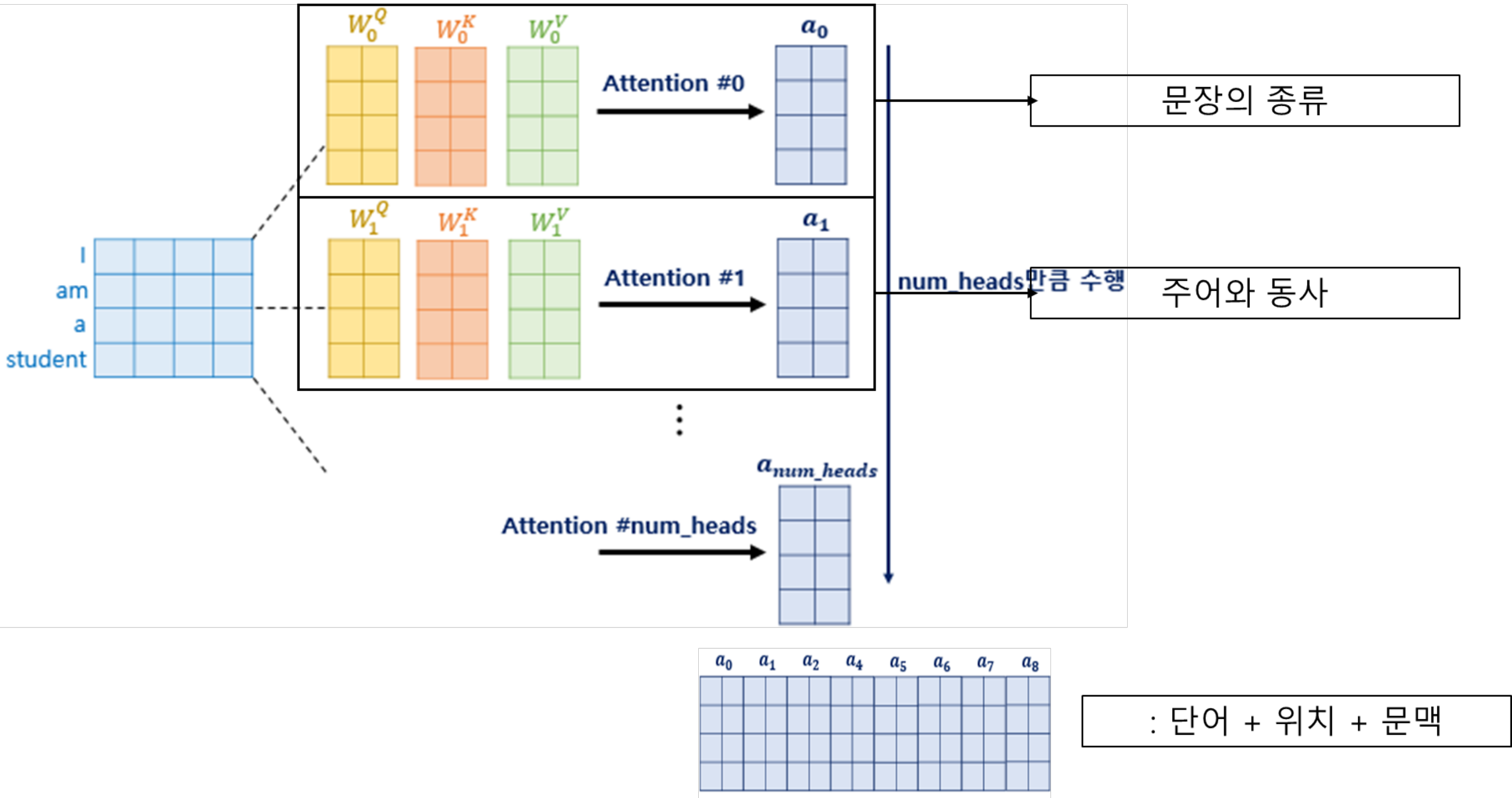
$$S = \begin{bmatrix} s_{11} & s_{12} & s_{13} & s_{14} \\ s_{21} & s_{22} & s_{23} & s_{24} \\ s_{31} & s_{32} & s_{33} & s_{34} \\ s_{41} & s_{42} & s_{43} & s_{44} \end{bmatrix} + M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

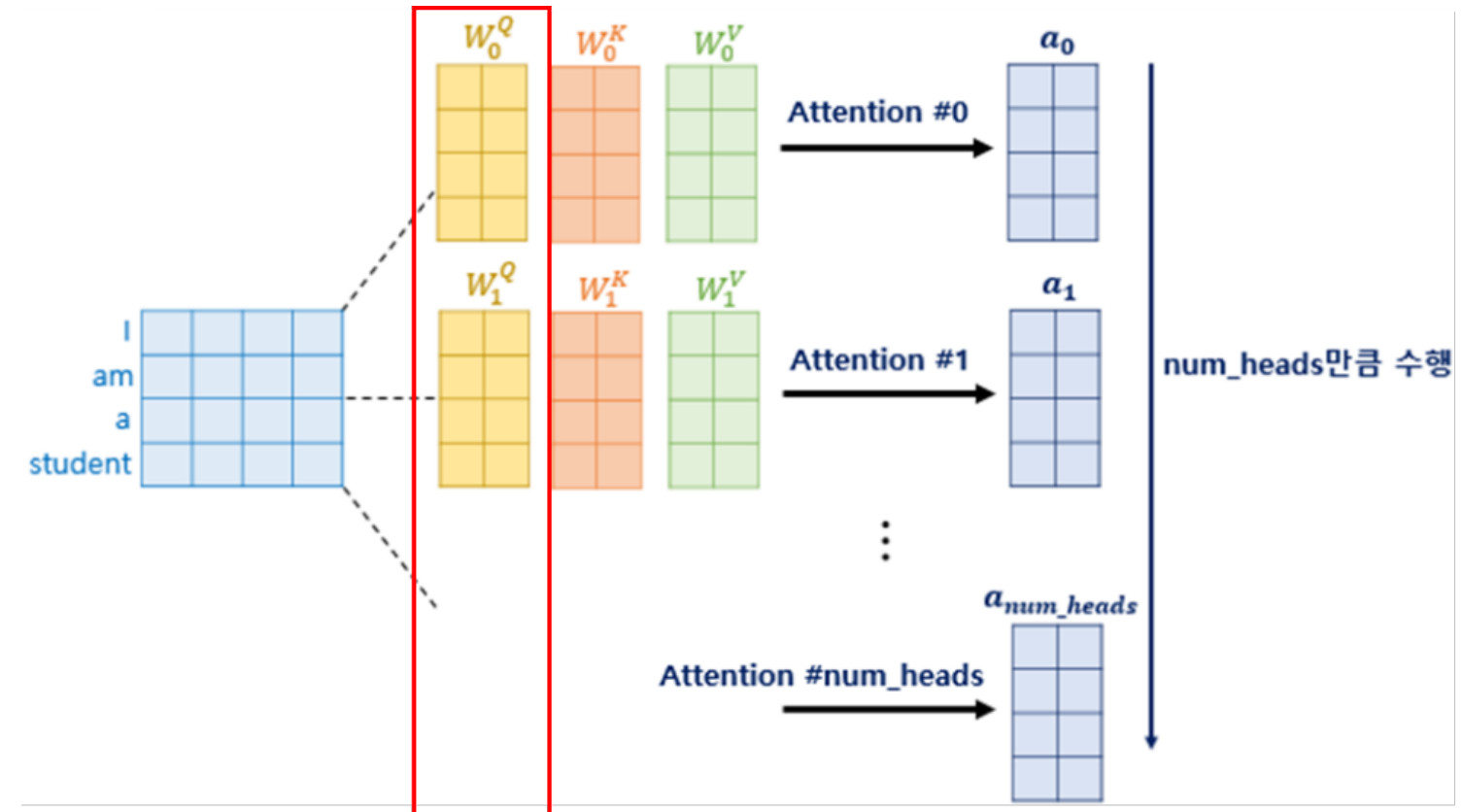
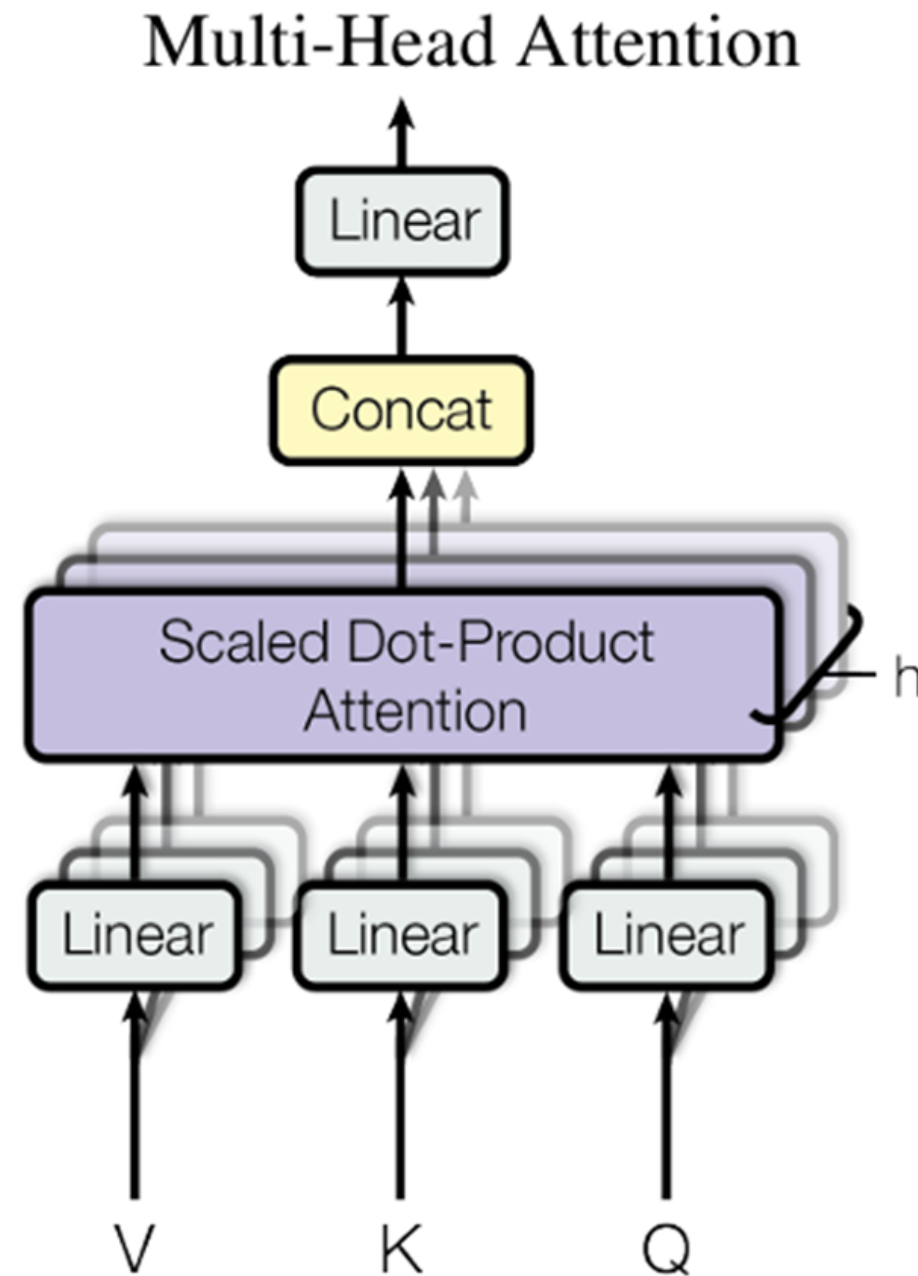
$$S_{\text{masked}} = \begin{bmatrix} s_{11} & -\infty & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty \\ s_{31} & s_{32} & s_{33} & -\infty \\ s_{41} & s_{42} & s_{43} & s_{44} \end{bmatrix}$$

$S_{\text{masked}} = S + M$

$$A = \begin{bmatrix} a_{11} & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

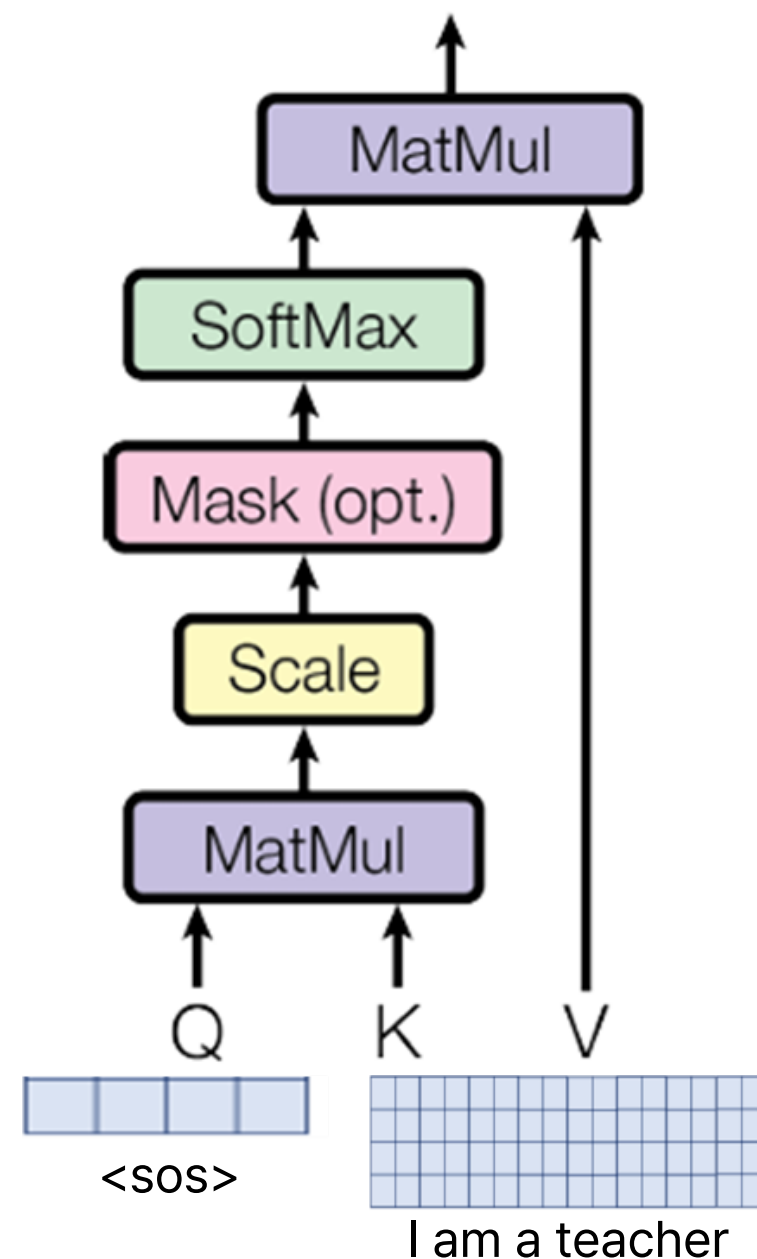
$A = \text{Softmax}(S_{\text{masked}})$



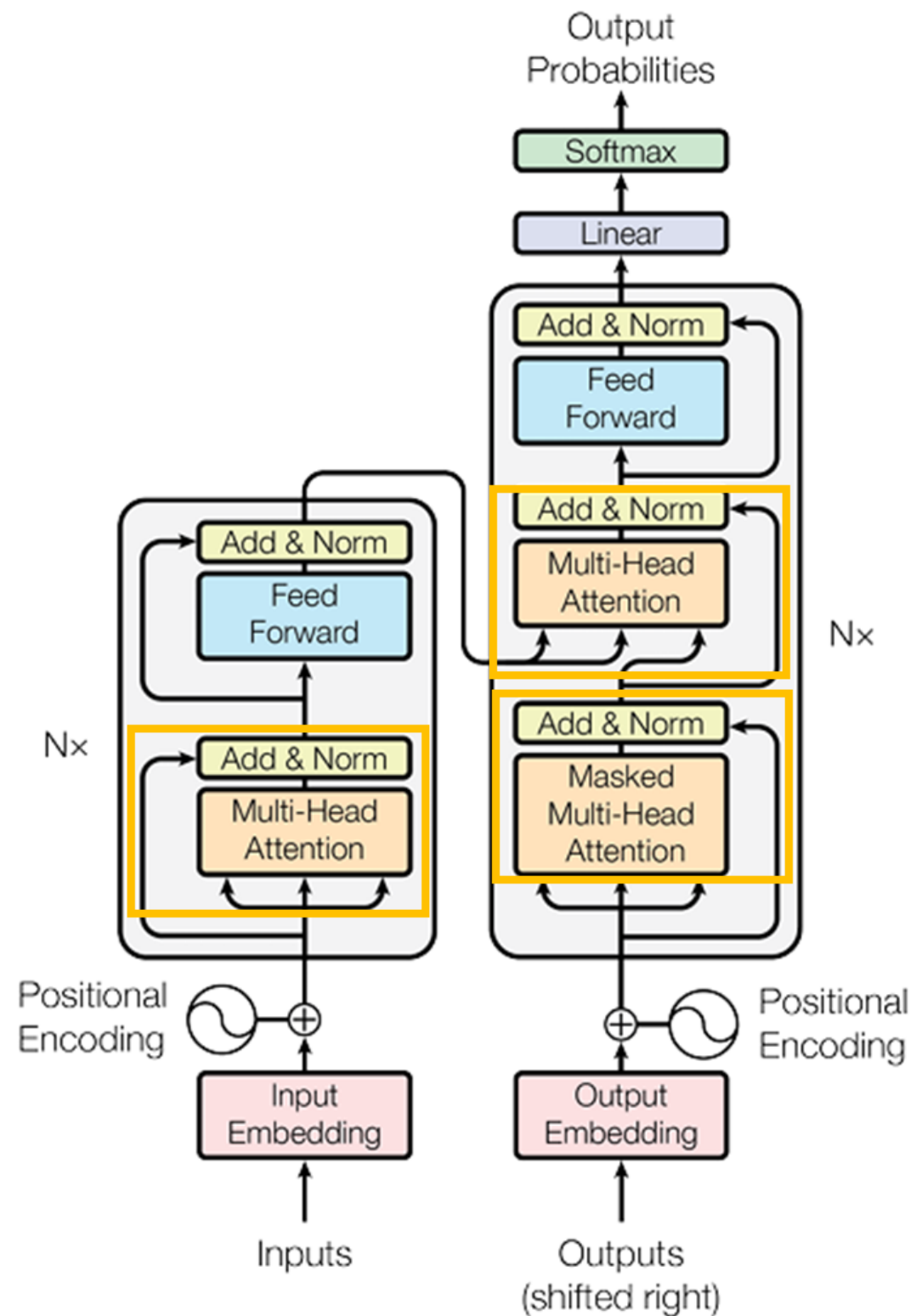


- 임베딩 차원(d_{model}) Q, K, V차원(d_{model}/h)
- X를 각각 서로 다른 가중치를 가진 선형 변환을 통해 $h(=8)$ 번 projection
- 각각 Attention을 병렬적으로 계산 후 도출된 Attention value들을 마지막에 concatenate를 통해 하나로 합침

Scaled Dot-Product Attention



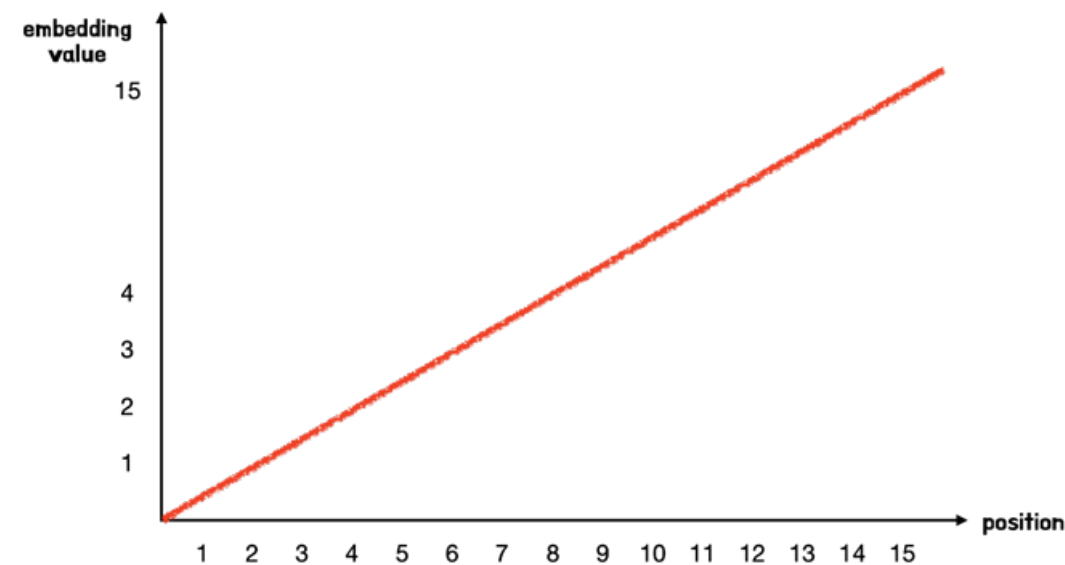
- 'I am a student ' 를 번역한다고 가정.
- Q = 디코더의 현재 상태 (현재까지 예측된 단어들)
- K, V = 인코더의 출력 (입력 문장 "I am a student"의 최종 표현)
- <sos>토큰이 embedding 되어 Self Attention을 거친 후 Q벡터가 됨
- Q와 K의 행렬곱 연산에 의해 <sos>토큰과 각각의 입력 단어("I", "am", "a", "teacher") 간의 연관성을 계산함
- Softmax를 적용하면 확률값이 되고, 각 입력 단어가 얼마나 중요한지를 나타냄.
- 어텐션 가중치를 V에 곱하면 최종적으로 <sos>가 입력되었을 때, 입력 문장 전체를 참고하여 "나는"을 예측할 최종 벡터를 생성함.
- 이후에는 Auto-Regressive 방식으로 디코더가 계속해서 이전 단어들을 입력으로 받아가면서 다음 단어를 예측하는 작업을 반복하게 됨



- Self-Attention (인코더에서 사용)
 - 입력 문장의 단어들 간의 관계를 학습
 - 어텐션을 수행하면 각 단어가 다른 단어들과 얼마나 관련이 있는지를 나타내는 가중치 행렬이 생성됨.
 - 결과적으로, 입력 문장은 단어 간의 관계(Context)가 반영된 새로운 표현(벡터)로 변환됨.
- Masked Self-Attention (디코더에서 사용)
 - Auto-Regressive 하게 이전 단어들만 참고하여 단어 생성이 가능하도록 마스킹을 적용
- Cross-Attention (디코더에서 사용)
 - 디코더가 인코더의 정보를 참고할 수 있도록 함

1. 정수 (첫 번째 토큰에는 1, 두 번째 토큰은 2....)

E_0	p_0	E_1	p_1	E_2	p_2	E_3	p_3	...	E_{14}	p_{14}
0.19	1	0.70	2	0.34	3	0.69	4		0.00	15
-0.47	1	-0.65	2	0.87	3	0.79	4		0.81	15
-0.77	1	0.11	2	-0.39	3	-0.25	4		0.31	15
0.59	1	0.04	2	-0.91	3	0.44	4		0.15	15



→ 시퀀스가 길어지면 단어 정보보다 위치 정보가 지배적이게 됨
→ 단어의 의미 훼손

2. 첫 번째 토큰에는 0, 마지막 토큰에는 1을 부과하고, 그 사이를 (1/시퀀스 길이)로 적용

E_0	p_0	E_1	p_1	E_2	p_2	E_3	p_3	...	E_8	p_8
0.19	0	0.70	0.33	0.34	0.66	0.69	1		0.69	1
-0.47	0	-0.65	0.33	0.87	0.66	0.79	1		0.79	1
-0.77	0	0.11	0.33	-0.39	0.66	-0.25	1		-0.25	1
0.59	0	0.04	0.33	-0.91	0.66	0.44	1		0.44	1

E_0	p_0	E_1	p_1	E_2	p_2	...	E_8	p_8
0.19	0	0.70	0.125	0.34	0.25		0.69	1
-0.47	0	-0.65	0.125	0.87	0.25		0.79	1
-0.77	0	0.11	0.125	-0.39	0.25		-0.25	1
0.59	0	0.04	0.125	-0.91	0.25		0.44	1

→ 시퀀스가 길이에 따라 같은 위치 정보에 해당하는 위치 벡터 값이 달라짐

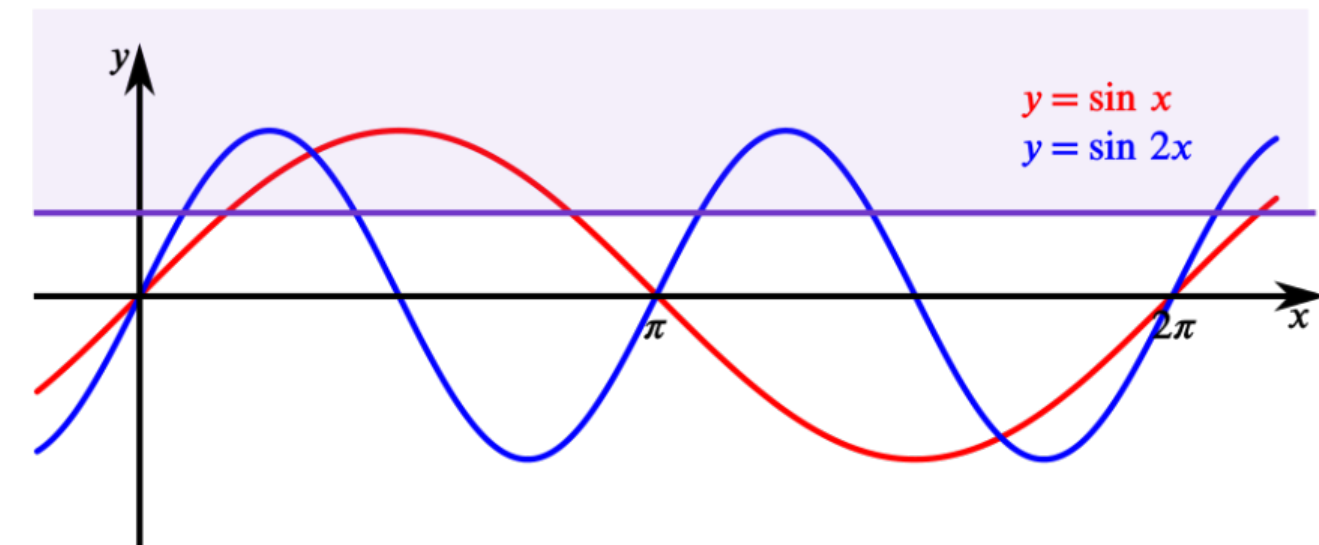
- 단어의 의미정보가 변질되지 않도록
- 같은 위치의 토큰은 항상 같은 위치 벡터값을 가지고 있도록

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- 범위가 -1~1로 안정적
- 주기함수로 글자수에 상관없음
- sin과 cos을 번갈아 사용하며 경우의 수를 증가시킴
- d_{model} 로 나눠 줘서 정규화를 해 줌 (분모가 너무 커져서 0이 됨을 방지)

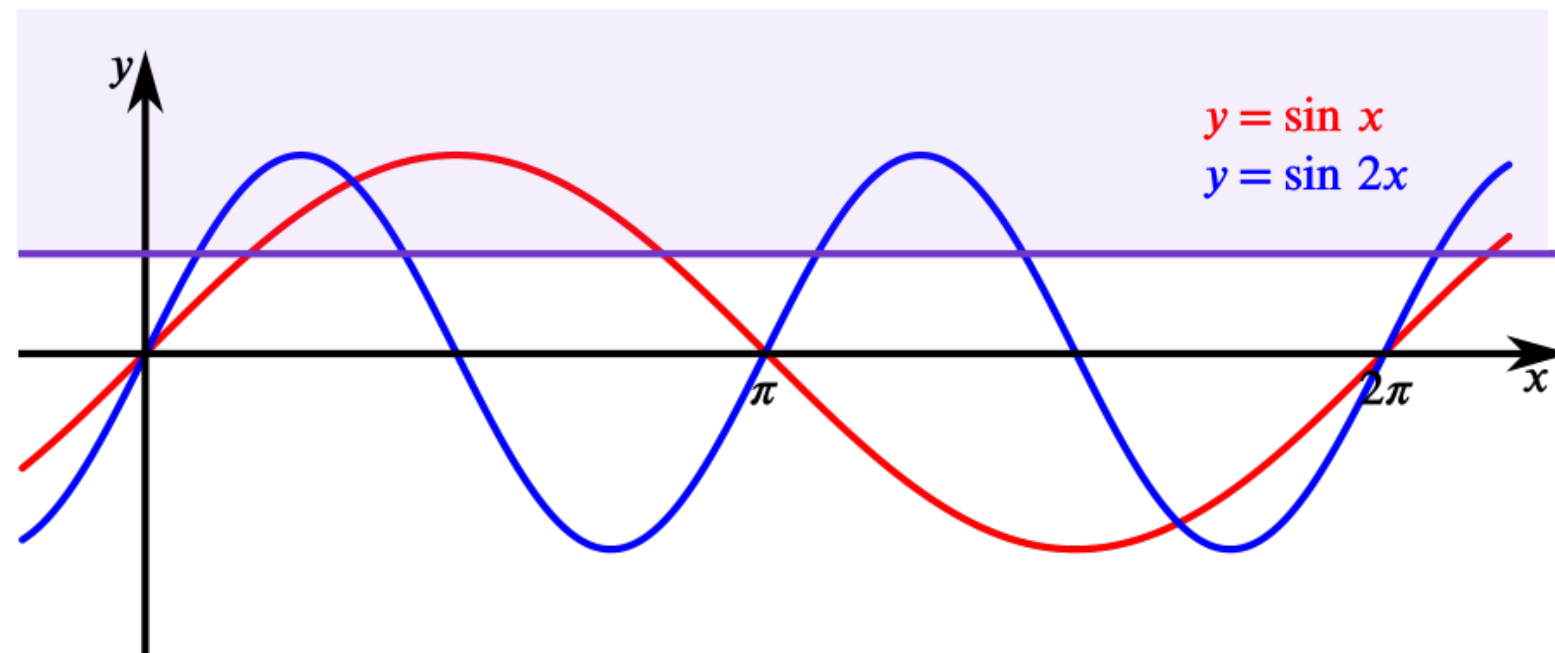
	i	0	1	2	3
I		1	1	1	1
am		1	1	1	0
student		1	1	0	0
		sin	cos	sin	cos



- 왜 값이 겹치지 않을까?

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



- pos = 단어의 위치 (0, 1, 2, ... 문장 길이)
- $d_{\text{model}} = 512$
- i = 차원의 인덱스 (0부터 시작)
- 낮은 차원(low index, 작은 i)
 - ➔ sin, cos의 주파수가 느림 (큰 값으로 나뉘어서 변동 폭이 작음)
 - ➔ 멀리 떨어진 위치들끼리도 비슷한 값을 가짐
- 높은 차원(high index, 큰 i)
 - ➔ sin, cos의 주파수가 빠름 (작은 값으로 나뉘어서 변동 폭이 큼)
 - ➔ 가까운 위치들도 완전히 다른 값을 가짐
- 결과:
 - 낮은 차원은 전반적인 위치(큰 범위)를 인식
 - 높은 차원은 세부적인 차이를 인식
 - 모든 차원을 종합하면 각 위치마다 고유한 벡터를 가질 수 있음!

- 최초의 완전 어텐션 기반(Attention-based) 시퀀스 변환 모델
 - RNN 없이도 자연어 처리(NLP)에서 최고의 성능을 달성
 - 병렬 연산 가능 기존 Seq2Seq 모델보다 학습 속도가 빠름
 - 멀티헤드 어텐션(Multi-Head Attention)을 통해 문맥을 더 잘 이해
-
- Limitation
 - $O(n^2)$ 연산량 → 긴 문장 처리 시 비효율적
 - Self-Attention이 지역 정보(Locality)를 잘 반영하지 못함
 - 순차 데이터(예: 음성, 시계열) 처리에 최적화되어 있지 않음



감사합니다